

Zadania

2.1 Pod adresem <http://127.0.0.1:2001> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2001/encrypt> oraz <http://127.0.0.1:2001/submit>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2001:2001 --name ex1 docker.io/mazurkatarzyna/symmetric-enc-ex1:latest
podman run -p 2001:2001 --name ex1 docker.io/mazurkatarzyna/symmetric-enc-ex1:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2001:2001 --name ex1 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex1:latest
podman run -p 2001:2001 --name ex1 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex1:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:2001/encrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`), losowe słowo do zaszyfrowania (`word`) oraz klucz szyfrujący podany jako 32-bajtowy ciąg szesnastkowy (`key_hex`).
- (c) Zaszyfruj słowo odebrane od serwera użyciu algorytmu AES-256 w trybie ECB wykorzystując odebrany od serwera klucz szyfrujący.
- (d) Po wykonaniu szyfrowania, zaszyfrowany ciąg znaków zakoduj w formacie `base64`.
- (e) Po uzyskaniu zaszyfrowanego i zakodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2001/submit> używając metody HTTP POST, w którym przekażesz zarówno identyfikator sesji (jako `session_id`), jak i zaszyfrowany ciąg w formacie `base64` (jako `encrypted_b64`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby zaszyfrować wylosowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

```

[user@parrot]-[~]
→ $curl -X 'GET' \
  'http://127.0.0.1:2001/encrypt' \
  -H 'accept: application/json'
{"key_hex":"12001e18414ce8b3c914d1bdee730d20b7b9dc194e5721b73e91d1ae3f393d53","session_id":"2e65489e74a1ed35","word":"15220"}
[user@parrot]-[~]
→ $echo -n "15220" > 2_1.txt
[user@parrot]-[~]
→ $openssl enc -aes-256-ecb -in 2_1.txt -K 12001e18414ce8b3c914d1bdee730d20b7b9dc194e5721b73e91d1ae3f393d53 -out 2_1.enc -base64
[user@parrot]-[~]
→ $cat 2_1.enc
dYAf1ysmAtNE05DaAEro0g==
[user@parrot]-[~]
→ $curl -X 'POST' \
  'http://127.0.0.1:2001/submit' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "session_id": "2e65489e74a1ed35",
    "encrypted_b64": "dYAf1ysmAtNE05DaAEro0g=="
  }'
{"result":"Correct encryption!"}

```

2.2 Pod adresem <http://127.0.0.1:2002> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2002/decrypt> oraz <http://127.0.0.1:2002/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 2002:2002 --name ex2 docker.io/mazurkatarzyna/symmetric-enc-ex2:latest
podman run -p 2002:2002 --name ex2 docker.io/mazurkatarzyna/symmetric-enc-ex2:latest

```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 2002:2002 --name ex2 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex2:latest
podman run -p 2002:2002 --name ex2 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex2:latest

```

- Wyślij request do endpointa <http://127.0.0.1:2002/decrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`), zakodowane i zaszyfrowane słowo (`encrypted_b64`) oraz klucz deszyfrujący podany jako 32-bajtowy ciąg szesnastkowy (`key_hex`).
- Odkoduj otrzymane słowo, stosując kodowanie `base64`. Następnie odszyfruj odkodowane słowo algorytmem AES-256 w trybie ECB z użyciem podanego klucza deszyfrującego.
- Po uzyskaniu odszyfrowanego i odkodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2002/submit> używając metody HTTP POST, w którym przekażesz zarówno identyfikator sesji (jako `session_id`), jak i odkodowane i odszyfrowane słowo (jako `decrypted_word`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.

```

[user@parrot]-[~]
└─$ curl -X 'GET' 'http://localhost:2002/decrypt' -H 'accept: application/json'
{"encrypted_b64":"DUeGWT9PV7dMITssehbpvA==","key_hex":"1ee8b237876db96edb3fb0f9c91a91edcb6ac55cca6eb76448c1cd1c18ae5110","session_id":"60789492d46f4404"}
[user@parrot]-[~]
└─$ openssl enc -d -aes-256-ecb -in <(echo DUeGWT9PV7dMITssehbpvA==) -out 2_2.dec -K 1ee8b237876db96edb3fb0f9c91a91edcb6ac55cca6eb76448c1cd1c18ae5110 -base64
[user@parrot]-[~]
└─$ cat 2_2.dec
singer67
[user@parrot]-[~]
└─$ curl -X 'POST' \
  'http://localhost:2002/submit' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "session_id": "60789492d46f4404",
    "decrypted_word": "singer67"
  }'
{"result":"Correct decryption!"}

```

2.3 Pod adresem <http://127.0.0.1:2003> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2003/encrypt> oraz <http://127.0.0.1:2003/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 2003:2003 --name ex3 docker.io/mazurkatarzyna/symmetric-enc-ex3:latest
podman run -p 2003:2003 --name ex3 docker.io/mazurkatarzyna/symmetric-enc-ex3:latest

```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 2003:2003 --name ex3 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex3:latest
podman run -p 2003:2003 --name ex3 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex3:latest

```

- (b) Wyślij request do endpointa <http://127.0.0.1:2003/encrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`), losowe słowo do zaszyfrowania (`word`) oraz klucz szyfrujący podany jako 32-bajtowy ciąg szesnastkowy (`key_hex`).
- (c) Zaszyfruj otrzymane od serwera słowo, stosując algorytm CAMELLIA-128 w trybie ECB wykorzystując odebrany od serwera klucz szyfrujący.
- (d) Po wykonaniu szyfrowania, zaszyfrowany ciąg znaków zakoduj w formacie `base64`.
- (e) Po uzyskaniu zaszyfrowanego i zakodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2003/submit> używając metody HTTP POST, w którym przekażesz zarówno identyfikator sesji (jako `session_id`), jak i zaszyfrowany ciąg w formacie `base64` (jako `encrypted_b64`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby zaszyfrować wylosowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

Przy tym <(echo ...) DAĆ -n czyli --> <(echo -n ...)

```
[user@parrot]-[~]
→ $curl -X 'GET' 'http://localhost:2003/encrypt' -H 'accept: application/json'
{"key_hex":"1f1aac1a8abfc8c1294a1826ebba55a9","session_id":"cf8fc677ee593376","word":"unloved13"}
[user@parrot]-[~]
→ $openssl enc -camellia-128-ecb -in <(echo -n unloved13) -K 1f1aac1a8abfc8c1294a1826ebba55a9 -out 2_3.enc -base64
[user@parrot]-[~]
→ $cat 2_3.enc
b1NyV5b51E0CZZKjAB5zCQ==
[user@parrot]-[~]
→ $curl -X 'POST' \
  'http://localhost:2003/submit' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "session_id": "cf8fc677ee593376",
    "encrypted_b64": "b1NyV5b51E0CZZKjAB5zCQ=="
  }'
{"result":"Correct encryption!"}
```

2.4 Pod adresem <http://127.0.0.1:2004> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2004/decrypt> oraz <http://127.0.0.1:2004/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2004:2004 --name ex4 docker.io/mazurkatarzyna/symmetric-enc-ex4:latest
podman run -p 2004:2004 --name ex4 docker.io/mazurkatarzyna/symmetric-enc-ex4:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2004:2004 --name ex4 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex4:latest
podman run -p 2004:2004 --name ex4 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex4:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:2004/decrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`), zakodowane i zaszyfrowane słowo (`encrypted_b64`) oraz klucz deszyfrujący podany jako 32-bajtowy ciąg szesnastkowy (`key_hex`).
- (c) Odkoduj otrzymane słowo, stosując kodowanie `base64`. Następnie odszyfruj odkodowane słowo algorytmem CAMELLIA-128 w trybie ECB z użyciem podanego klucza deszyfrującego.
- (d) Po uzyskaniu odszyfrowanego i odkodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2004/submit> używając metody HTTP POST, w którym przekażesz zarówno identyfikator sesji (jako `session_id`), jak i odkodowane i odszyfrowane słowo (jako `decrypted_word`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.

```

[user@parrot]-[~]
└─$ curl -X 'GET' 'http://localhost:2004/decrypt' -H 'accept: application/json'
{"encrypted_b64": "fcnPIPJc0QRBwf637sG2Ug==", "key_hex": "803911e2f925b306b2f8e12f5a6f90cb", "session_id": "ebe20a3eaa302616"}
[user@parrot]-[~]
└─$ openssl enc -d -camellia-128-ecb -in <(echo fcnPIPJc0QRBwf637sG2Ug==) -out 2_4.dec -K 803911e2f925b306b2f8e12f5a6f90cb -base64
[user@parrot]-[~]
└─$ cat 2_4.dec
mogily
mogily [user@parrot]-[~]
└─$ curl -X 'POST' \
'http://localhost:2004/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "ebe20a3eaa302616",
  "decrypted_word": "mogily"
}'
{"result": "Correct decryption!"}

```

2.5 Pod adresem <http://127.0.0.1:2005> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2005/encrypt> oraz <http://127.0.0.1:2005/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 2005:2005 --name ex5 docker.io/mazurkatarzyna/symmetric-enc-ex5:latest
podman run -p 2005:2005 --name ex5 docker.io/mazurkatarzyna/symmetric-enc-ex5:latest

```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 2005:2005 --name ex5 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex5:latest
podman run -p 2005:2005 --name ex5 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex5:latest

```

- Wyślij request do endpointa <http://127.0.0.1:2005/encrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`) oraz losowe słowo do zaszyfrowania (`word`).
- Wygeneruj klucz szyfrujący (klucz musi mieć 192 bity).
- Zaszyfruj otrzymane od serwera słowo, stosując algorytm ARIA-192 w trybie ECB wykorzystując wygenerowany przez Ciebie (w punkcie 2.5c) klucz szyfrujący.
- Po wykonaniu szyfrowania zaszyfrowany ciąg znaków zakoduj w formacie `base64`.
- Po uzyskaniu zaszyfrowanego i zakodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2005/submit> używając metody HTTP POST, w którym przekażesz: identyfikator sesji (`session_id`), zaszyfrowany ciąg w formacie `base64` (`encrypted_b64`), wygenerowany przez Ciebie klucz szyfrujący w formacie szesnastkowym (`key_hex`, 192 bity). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby zaszyfrować wylosowane przez serwer słowo i wygenerować klucz szyfrujący wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

Informacja do obliczeń

```
w openssl rand -hex 24 > key
```

24, ponieważ $\rightarrow 192 / 8 = 24$

```
[user@parrot]-[~]
└─$ curl -X 'GET' \
  'http://localhost:2005/encrypt' \
  -H 'accept: application/json'
{"session_id":"6d42730faabc37eb","word":"LAWRENCE21"}
[user@parrot]-[~]
└─$ openssl rand -hex 24 > key
[user@parrot]-[~]
└─$ cat key
a88b1706134ccef10177f9f278d74b89b0d528b86b1d706d
[user@parrot]-[~]
└─$ openssl enc -aria-192-ecb -in <(echo -n LAWRENCE21) -K $(cat key) -out 2_5.enc -base64
[user@parrot]-[~]
└─$ cat 2_5.enc
cZt8m3r1Tqwp4kHtk0z2ig==
[user@parrot]-[~]
└─$ curl -X 'POST' \
  'http://localhost:2005/submit' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "session_id": "6d42730faabc37eb",
    "encrypted_b64": "cZt8m3r1Tqwp4kHtk0z2ig==",
    "key_hex": "a88b1706134ccef10177f9f278d74b89b0d528b86b1d706d"
  }'
{"result":"Correct encryption!"}
```

2.6 Pod adresem <http://127.0.0.1:2006> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2006/encrypt> oraz <http://127.0.0.1:2006/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2006:2006 --name ex6 docker.io/mazurkatarzyna/symmetric-enc-ex6:latest
podman run -p 2006:2006 --name ex6 docker.io/mazurkatarzyna/symmetric-enc-ex6:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2006:2006 --name ex6 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex6:latest
podman run -p 2006:2006 --name ex6 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex6:latest
```

- (b) Wyślij żądanie HTTP GET do endpointa <http://127.0.0.1:2006/encrypt>. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`) oraz losowe słowo do zaszyfrowania (`word`).
- (c) Wygeneruj klucz szyfrujący (klucz musi mieć 128 bitów).
- (d) Wygeneruj wektor inicjalizacyjny (IV) (klucz musi mieć 128 bitów).
- (e) Zaszyfruj pobrane od serwera słowo przy użyciu algorytmu AES-128 w trybie CBC, z kluczem szyfrującym oraz wektorem inicjalizującym (IV) wygenerowanymi przez Ciebie (w punktach **2.6c** i **2.6d**).
- (f) Po wykonaniu szyfrowania zakoduj zaszyfrowany ciąg do formatu `base64`.
- (g) Po uzyskaniu zaszyfrowanego i zakodowanego wyniku wyślij go do serwera poprzez endpoint <http://127.0.0.1:2006/submit> używając metody HTTP POST, przekazując następujące dane: identyfikator sesji (`session_id`), zaszyfrowany ciąg w formacie `base64` (`encrypted_b64`), klucz szyfrujący w formacie szesnastkowym (`key_hex`, 128 bitów) oraz IV w formacie szesnastkowym (`iv_hex`, 128 bitów). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby zaszyfrować wylosowane przez serwer słowo, wygenerować klucz szyfrujący oraz IV, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

Informacja do obliczeń

```
w openssl rand -hex 16 > key
```

16, ponieważ $\rightarrow 128 / 8 = 16$


```

[user@parrot]-[~]
→ $curl -X 'GET' \
  'http://localhost:2006/encrypt' \
  -H 'accept: application/json' \
  '{"session_id":"c9927030c7db642f","word":"babypoop"}'
[user@parrot]-[~]
→ $openssl rand -hex 16 > key
[user@parrot]-[~]
→ $cat key
b8836d5ce310eda30482eec99032f38e
[user@parrot]-[~]
→ $openssl rand -hex 16 > IV
[user@parrot]-[~]
→ $cat IV
b06b4d0b2a8ca73ef844f723bca487c5
[user@parrot]-[~]
→ $openssl enc -aes-128-cbc -in <(echo -n babypoop) -out 2_6.enc -K $(cat key) -iv $(cat IV) -no
salt -base64
[user@parrot]-[~]
→ $cat 2_6.enc
kqZ/UuGnDpQdXAH1Qu2Adg==
[user@parrot]-[~]
→ $curl -X 'POST' \
  'http://localhost:2006/submit' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "session_id": "c9927030c7db642f",
    "encrypted_b64": "kqZ/UuGnDpQdXAH1Qu2Adg==",
    "key_hex": "b8836d5ce310eda30482eec99032f38e",
    "iv_hex": "b06b4d0b2a8ca73ef844f723bca487c5"
  }'
{"result":"Correct encryption!"}

```

2.7 Pod adresem <http://127.0.0.1:2007> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2007/decrypt> oraz <http://127.0.0.1:2007/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 2007:2007 --name ex7 docker.io/mazurkatarzyna/symmetric-enc-ex7:latest
podman run -p 2007:2007 --name ex7 docker.io/mazurkatarzyna/symmetric-enc-ex7:latest

```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 2007:2007 --name ex7 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex7:latest
podman run -p 2007:2007 --name ex7 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex7:latest

```

(b) Wyślij żądanie HTTP GET do endpointa <http://127.0.0.1:2007/decrypt>. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), zakodowany w formacie base64 ciąg zaszyfowanego tekstu (`encrypted_b64`), użyte hasło (`password`) oraz IV w formacie szesnastkowym (`iv_hex`). Serwer zaszyfował losowo wybrane słowo przy użyciu algorytmu AES-256-CBC, z kluczem wygenerowanym na podstawie hasła przy użyciu funkcji PBKDF2, oraz losowego wektora inicjalizującego (IV).

- (c) Odkoduj otrzymane słowo, stosując kodowanie `base64`. Następnie odszyfruj odkodowane słowo algorytmem AES-256-CBC, z użyciem kluczem wygenerowanym na podstawie hasła przy użyciu funkcji PBKDF2, oraz losowego wektora inicjalizującego (IV).
- (d) Pamiętaj, że do szyfrowania NIE użyto soli, więc w deszyfrowaniu również NIE JEST potrzebna.
- (e) Wyślij do serwera odpowiedź poprzez endpoint `http://127.0.0.1:2007/submit`, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`) oraz odszyfrowane słowo (`decrypted_word`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.

```
[user@parrot]~$ curl -X 'GET' \
'http://localhost:2007/decrypt' \
-H 'accept: application/json'
{"encrypted_b64": "Qn88050G49KzmRtadPdE99o7CoisRhR6leFZXVjR508=", "iv_hex": "dcc5b9e5d485e08cf1138f77b1aeebf8", "password": "610831", "session_id": "53fe9c4e9bf65db0"}
```

```
[x]-[user@parrot]~$ openssl enc -d -aes-256-cbc -pbkdf2 -pass pass:610831 -in <(echo Qn88050G49KzmRtadPdE99o7CoisRhR6leFZXVjR508=) -out 2_7.dec -base64 -iv dcc5b9e5d485e08cf1138f77b1aeebf8 -nosalt
[user@parrot]~$ cat 2_7.dec
mesha13
```

```
$curl -X 'POST' \
'http://localhost:2007/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "53fe9c4e9bf65db0",
  "decrypted_word": "mesha13"
}'
{"result": "Correct decryption!"}
```

2.8 Pod adresem <http://127.0.0.1:2008> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2008/decrypt> oraz <http://127.0.0.1:2008/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2008:2008 --name ex8 docker.io/mazurkatarzyna/symmetric-enc-ex8:latest
podman run -p 2008:2008 --name ex8 docker.io/mazurkatarzyna/symmetric-enc-ex8:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2008:2008 --name ex8 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex8:latest
podman run -p 2008:2008 --name ex8 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex8:latest
```

- (b) Wyślij żądanie HTTP GET do endpointa <http://127.0.0.1:2008/decrypt>. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), zakodowany w formacie `base64` ciąg zaszyfrowane, losowe słowo (`encrypted_b64`), użyte hasło (`password`) oraz IV w formacie szesnastkowym (`iv_hex`). Serwer zaszyfrował losowo wybrane słowo przy użyciu algorytmu 3DES, z kluczem wygenerowanym na podstawie hasła przy użyciu funkcji PBKDF2, oraz losowego wektora inicjalizującego (IV).
- (c) Odkoduj otrzymane słowo, stosując kodowanie `base64`. Następnie, odszyfruj otrzymany ciąg, stosując algorytm 3DES, funkcję PBKDF2 do wyprowadzenia klucza z hasła oraz przekazany IV.
- (d) Pamiętaj, że do szyfrowania NIE użyto soli, więc w deszyfrowaniu również NIE JEST potrzebna.
- (e) Wyślij do serwera odpowiedź poprzez endpoint <http://127.0.0.1:2008/submit>, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`) oraz odszyfrowane słowo (`decrypted_word`). Serwer w odpowiedzi zwróci odpowiednią informację o sukcesie lub błędzie.

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

```
[user@parrot]~$ curl -X 'GET' 'http://localhost:2008/decrypt' -H 'accept: application/json'
{"encrypted_b64":"6Q0SrnRNrrHd7U5ch3Sr027EVMCVL0h3","iv_hex":"56aebbe87c67ad73","password":"annine","session_id":"161a7a25eb71bf55"}
```

```
[x]~[user@parrot]~$ openssl enc -d -des-ede3-cbc -pbkdf2 -pass pass:annine -in <(echo 6Q0SrnRNrrHd7U5ch3Sr027EVMCVL0h3) -out 2_8.dec -base64 -iv 56aebbe87c67ad73 -nosalt
[user@parrot]~$ cat 2_8.dec
jerez2
```

```
[user@parrot]~$ curl -X 'POST' \
'http://localhost:2008/submit' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "session_id": "161a7a25eb71bf55",
  "decrypted_word": "jerez2"
}'
{"result": "Correct decryption!"}
```

2.9 Pod adresem <http://127.0.0.1:2009> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2009/decrypt> oraz <http://127.0.0.1:2009/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2009:2009 --name ex9 docker.io/mazurkatarzyna/symmetric-enc-ex9:latest
podman run -p 2009:2009 --name ex9 docker.io/mazurkatarzyna/symmetric-enc-ex9:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2009:2009 --name ex9 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex9:latest
podman run -p 2009:2009 --name ex9 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex9:latest
```

- (b) Wządanie HTTP GET do endpointa <http://127.0.0.1:2009/decrypt>. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), zakodowany w formacie `base64` ciąg zaszyfrowane, losowe słowo (`encrypted_b64`) oraz użyte hasło (`password`). Serwer zaszyfrował losowo wybrane słowo przy użyciu algorytmu AES-256-ECB, z kluczem wygenerowanym na podstawie hasła przy użyciu funkcji PBKDF2 z liczbą iteracji równą 356.
- (c) Odkoduj otrzymane słowo, stosując kodowanie `base64`. Następnie, odszyfruj otrzymany ciąg, stosując algorytm AES-256-ECB, z kluczem wygenerowanym na podstawie hasła przy użyciu funkcji PBKDF2 z liczbą iteracji równą 356.
- (d) Pamiętaj, że do szyfrowania NIE użyto soli, więc w deszyfrowaniu również NIE JEST potrzebna.
- (e) Wyślij do serwera odpowiedź poprzez endpoint <http://127.0.0.1:2009/submit>, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`) oraz odszyfrowane słowo (`decrypted_word`).

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.

```
[user@parrot]~  
$ curl -X GET http://localhost:2009/decrypt  
{ "encrypted_b64": "6olX8il9mJH5g6eXIIGaqeK5SdGIi//wQhqkbQSizrM=", "password": "danger  
dog", "session_id": "9f151eb58be0017f" }
```

```
[x]-[user@parrot]~  
$ echo -n "6olX8il9mJH5g6eXIIGaqeK5SdGIi//wQhqkbQSizrM=" | base64 -d > encrypted.  
bin
```

```
[x]-[user@parrot]~  
$ openssl enc -d -aes-256-ecb -in encrypted.bin -out 3_9.dec -pass pass:danger  
dog -pbkdf2 -iter 356 -nosalt
```

```
[user@parrot]~  
$ cat 3_9.dec  
garsoon13 [user@par
```

```
[user@parrot]~  
$ curl -X POST http://localhost:2009/submit -H 'Content-Type: application/json'  
' -d '{ "session_id": "9f151eb58be0017f", "decrypted_word": "garsoon13" }'  
{ "result": "Correct decryption!" }
```

Wyjaśnienie komendy openssl:

- **-d**: Określa, że operacja jest odszyfrowaniem.
 - **-aes-256-ecb**: Określa użycie algorytmu AES z 256-bitowym kluczem w trybie ECB.
 - **-in encrypted.bin**: Określa plik wejściowy, który zawiera zaszyfrowane dane.
 - **-out decrypted.txt**: Określa plik wyjściowy, w którym zapisane będą odszyfrowane dane.
 - **-pass pass:YOUR_PASSWORD**: Określa hasło używane do generowania klucza (może to być hasło podane w odpowiedzi).
 - **-pbkdf2**: Określa, że klucz będzie generowany za pomocą funkcji **PBKDF2**.
 - **-iter 356**: Określa liczbę iteracji funkcji PBKDF2 (w tym przypadku 356, jak wymagano).
 - **-nosalt**: do szyfrowania NIE u»yto soli, więc w deszyfrowaniu również NIE JEST potrzebna.
-

2.10 Pod adresem `http://127.0.0.1:2010` działa prosty serwer HTTP udostępniający 2 endpointy: `http://127.0.0.1:2010/decrypt` oraz `http://127.0.0.1:2010/submit`.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2010:2010 --name ex10 docker.io/mazurkatarzyna/symmetric-enc-ex10:latest
podman run -p 2010:2010 --name ex10 docker.io/mazurkatarzyna/symmetric-enc-ex10:latest
```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2010:2010 --name ex10 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex10:latest
podman run -p 2010:2010 --name ex10 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex10:latest
```

- (b) Wyślij żądanie HTTP GET do endpointa `http://127.0.0.1:2010/decrypt`. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), zakodowany w formacie `base64` ciąg zaszyfrowanego tekstu (`encrypted_b64`), klucz użyty do szyfrowania (`key_hex`) oraz IV (jeśli wylosowany algorytm go wymaga, `iv_hex`).
- (c) Twoim zadaniem jest odszyfrowanie zaszyfrowanego słowa przesłanego przez serwer. Serwer szyfruje losowo wybrane słowo przy użyciu losowego algorytmu dostępnego w bibliotece `OpenSSL`.
- (d) Odgadnij, jaki algorytm został wykorzystany do zaszyfrowania słowa, i odszyfruj otrzymany ciąg. (Pamiętaj, że po zaszyfrowaniu ciąg był zakodowany przy użyciu kodowania `base64`, więc najpierw odkoduj, a później odszyfruj ciąg.)
- (e) Wyślij do serwera odpowiedź poprzez endpoint `http://127.0.0.1:2010/submit`, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`), odszyfrowane słowo (`decrypted_word`) i znaleziony algorytm, którym słowo zostało zaszyfrowane (`algorithm`).

UWAGI:

- Aby odszyfrować zaszyfrowane przez serwer słowo, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.

```
[user@parrot]-[~]
→ $curl -X 'GET' \
'http://localhost:2010/decrypt' \
-H 'accept: application/json'
{"encrypted_b64": "mwrczLvLFLKa1wfE3FNwcQ==", "iv_hex": "f95187972b4d77235127452fb696c0e0", "key_hex": "
dd9fb6d059552b72cf3bbcdc5d019ebb", "session_id": "be5fe2de6d2a0ebe"}
[user@parrot]-[~]
→ $touch skrypt.py
[user@parrot]-[~]
→ $python3 skrypt.py
key=24B, iv=16B
CBC AES
```

```

import requests

r = requests.get("http://127.0.0.1:2010/decrypt").json()

key_hex = r["key_hex"]
iv_hex = r.get("iv_hex", "")

key_len = len(key_hex)//2
iv_len = len(iv_hex)//2

print(f"key={key_len}B, iv={iv_len}B")

if iv_len == 0:
    print("ECB mode")
elif iv_len == 16:
    print("CBC AES")
elif iv_len == 8:
    print("CBC DES/3DES")

```

DOKOŃCZYĆ - nie mam pojęcia jak zrobić

2.11 Pod adresem <http://127.0.0.1:2011> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2011/encrypt> oraz <http://127.0.0.1:2011/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 2011:2011 --name ex11 docker.io/mazurkatarzyna/symmetric-enc-ex11:latest
podman run -p 2011:2011 --name ex11 docker.io/mazurkatarzyna/symmetric-enc-ex11:latest

```

Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 2011:2011 --name ex11 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex11:latest
podman run -p 2011:2011 --name ex11 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex11:latest

```

- (b) Wyślij żądanie HTTP GET do endpointa <http://127.0.0.1:2011/encrypt>. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), obrazek do zaszyfrowania (`image_data_base64`), klucz do szyfrowania (`key_hex`) oraz IV (`iv`).
- (c) Do zaszyfrowania obrazka użyj algorytmu ARIA-128-CTR (bez paddingu), wykorzystując odebrany od serwera klucz szyfrujący oraz IV.
- (d) Wyślij do serwera odpowiedź poprzez endpoint <http://127.0.0.1:2011/submit>, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`) i zaszyfrowany obrazek zakodowany w `base64` (obrazek jest najpierw zaszyfrowany, a potem zakodowany) (jako `encrypted_data`). Serwer zwróci w odpowiedzi informację o tym, czy obrazek jest prawidłowo zaszyfrowany.

UWAGI:

- Aby zaszyfrować wylosowany przez serwer obrazek, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu HTTP `Content-Type: application/json`.


```
[user@parrot]~$ curl -s -o data.txt -X GET 'http://localhost:2011/encrypt'
```

```
home > user > data.txt
1 {
2   "algorithm": "ARIA-128-CTR",
3   "image_data_base64": "iVBORw0KGgoAAAANSUHEUgAAAEAAAABACAYAAACQaXHeAAANYU1EQVR4Xu2baY",
4   "iv": "3e7275b2c6d2c14fec87c685fb61814c",
5   "key": "e7b72ae8cbdbff7b4a4ab3a6c3ab0b7d",
6   "key_size_bits": 128,
7   "session_id": "07544d15-674c-4ff9-bf70-4e260a468712"
8 }
```

```
[user@parrot]~$ echo "iVBORw0KGgoAAAANSUHEUgAAAEAAAABACAYAAACQaXHeAAANYU1EQVR4Xu2baYxkV3XHf+
fet9Ta3d0zemzPPsZAYmdxHMTGQpAQFmEmyNZE2ECwUWQ2g0MUJYj4kTCKFENlkBCGADY5I4hhhI JhNKGyG
JBgirMEEbDwrPdN2z3iWXqa6q+ot95xoPGqV3N2FvPTYnQk/6ej2eyW13vmfv85797wq/p/zS4Q1xP6vvu
gCS/TlVsrFglyI01Uoy8AGLVhspmCaC7TMQktUxgUdBRT16G7xcn+7G3a9cPuD+f8ZAR68+4VJXB94q7jo
bQoXgCACaWxE7lQoThTMwBQzQ0tFFUJQisIoS8PUAAWzwqH/49Dvep0vrN+++zsi2JIUYpZ+3x08/lj76y
ZyaZoIg01HJRXSGAQFUzADnf27J8Tcc0VhdDMly6GTQTcTzBTBHvVmn/FpceuGqw4eZg4RzyFTJ6ZvMnOX
nrPCaNYBMRABA0MPExsjrhrNioIpGmCmAyfbcu5Ml3eHrn/bvk9v/OTRw8V7Ln/3Ix10g+M5RNUde2KyNl
vV3kmbq4QtII4xYfdGsxo4dzhnw8qcRiVUQsk7VqyKdh24a+PqJSHAof90bvfe7jjsyDu20gS0Q9KKXa++Y
3rm+oiSRcs5QwXnLC5xwYQj6qSUhwEt2fKdMhStFeHjsGGs5gvWivyD03GELJG0L06MalyQ+YGqXLQkBAM
5/xffHxRW/Z2aPjh0FUwNTmF3N+gfMdc8kTAADGFsIqFT0Isiu2XJCACw5eU/HPWu/MOyND0+oQt1/Tm2
t16Y9Um891kIwujxiHbmNML+eN01o+9fUgIAbH7lD7/lx06ZmhZC6FW/t9qcY/r0iF7is5V/dWiK1zuxd
6w+Q2HPkQPHEsI8/LXqkprhjn3/17yfYToxRyXjI17stIFj163+fUH7wJYsgJsfcUPdjnsoVbb5jTCPk2R
XmDM+/xEK2Km63GEmxdKHiBiieFFv9nNeIEGxQmAgc21twJznDCnR7Qzx4mWJ/L2b5tfd/BW+uBYygjhXl
OlM/VxQS/5vmEGj016BBuZLmbeSI+17wCl2IXF5L1RSxSM+TbnFzVHY7z1KIJYFNkfXXztYzPPym7w4S9d
ujGJk0vwfqtAQAE4kzYa9pDyw02/c99BniR7Pn9RZ6ihlZWdYVYAeqst7AAUzChLY+RogsCXtlyzfXvAGX
PAga9d8jxTeWvAbTNzGwo1sBIQwAiz1Wkre7506Yh3eo8EP5p2/276Q+YnQjBzkXn3N56QizQD3qNT40i
kfBnAGdEgP1ffuE602RnmbPNwFcqRq0qVFI18kIcXgUpVKWRjc3pmdsQzezmwS7ce8XLr7HReW7N1/54C
EWQLCWhtnk5jXCOQL0jvMCTrYdXspPbbzm4MNnRID9X7zorWUmt4inPlAPLB8UohgQBwiIgAFA4u3xqCXK
c020PU9M4U/0yFV1519x4AsveMum1z5050LbREF7ApjNe7qD+QJmzkSAZVHgfYs+ETJD9n3uBZ9U/PWVCq
wZNPJYQBxIb2zNQm+68zp5XsCRE0I3N/OUT2y+evfNvakN7Ln7+fSaFd18zrICjP4u6K2EIPz8WAKmd229
duRaniTrk07+8xfeqcY1A7WS1c0GOAfmAKUMjpnMm05AERYlCiBETok9NCpQT5XIGZiReOW8YeXopMjJtr
znwN3PWwm7r2cWVS8YvR6wUCNkTvUdqmapt1sBF1WafXdf8Ddqs2yZsnKQQMcmBIUxqc9U22HmiDORoTw
Y3FYfKAsbVVRcPFMRzY4PEP1wHDDcBh0jDvdAS/CRMTdt/8zm36y+Q807AQwM0F1Tg+gb0M0NabaHsH+c8
PrDv5oUQU4cPfWl5XB/rReUVY0FEzAICs8Yx0eshR1Xj+bIB/Yu02BB1iAkS9e/Oul6s0TJ7m6NePd0UOB
SqRgxopGIM89M133tz+/a8uujdfs+66gQ156AgBg/ZxuwvGVwYgj98FFGov3JrZJ6Dzonduyf1WBcwLia0
e0sYkUg7Gk4q7ec0VPvs+TY0SeX7k8L+yzgq5d05RTSwKYogoHj0aUwR7ohPaLKr7RwTEsZbgr5k6CFmyK
I8dSyiB7trz+4IUiGE8BR3+oausGDWxZ3shxKJhSlMb8QhM9yROL5mffH82/P5P76sk5W9htvfWuKcoDc
xwBIbrBab2awNx9R2YSeT67wShdzzTdeSl4EQ/1Et+kQQIpbxjZRMWs5eCEcmHBpsouJ49YarfnaYp8i6
bbvHPHa1Bps8MhHNjSvApSSN1KC8HTMiZ79wGzwbE9MeMT2m46N3ACyaAC0fXn+5qm0drJYIBqZMd6CTgU
N3nH/1z/byNNm0ffdu78JfdjKY7gBmCKdFw0x8MCIpwRZyQW/NCmhngnP6ia3vJFtUAYq8eA1m1JNiVnUm
Z07vsEZ37/0oz5BDDx/4C0jBU/9zNtFGWmKqLowAnSfCsSmPmE2lqXyQp01EP0R+O3FK7BRMCGp0MvDCF1
6yg/KZT4Qp999193Qy3qUKDiN2hgjEEEnAoWP/RdzsT2pkjinTn+dvHxhddAFpBEkcBTAfHNxfMDO/CV1gs
vH5FC97V7gr1pGf1ShzAtP8LEYzjrQri0DzR5haAXeAMhz73lazzKBXMKKXcyyLhs2x/kUeUnQAWyAKEYN
SS0Ks+813Q6kZ0c4i9vf+SG8baZ0YAt0LoOUAVUM00jR1lkfCV+IjNBMpMMRdoFSmCUY8LMGMeZpjB8ZbH
```



```
if18vHv4NoAzI4DRDYHarANmn8zc0NrVMHaIRSAfz86jcLhQoiWczCPqSunkt08bnxPTCUWJRb688ZIbKM
6gAOWJmIlgNU0CICIDhkS3AogiQOLkoLwJRCEXknqDGsloGpixEJxcmZjze2Z2b33T8qwBnTADB9heInD9r
xUoUEIyAvRr4NotA2S23EwRnxpRWaCQl1SiAMQfDtdGymYlpmKf+ThYJRx+c8Y08hKIEVPESqEYlobDX3r
uDiGfI3g+Tdtv2spSCY5KCBVY1MzCdH2oc0xlRBCz2+vaN141MnnEB1OKLmDHdkdkLYaiSYWYb1q1f/Xae
IWk5dGvZtWWu6SgQ1gx0iCQsM0lVWh1hsu2JnP3Txut03P0sfUlq721Du+0IC9YNdxAREGF0skq38J0R2G
Wb3nx8N0+DR3c2r5g6Kfcmy30cYmFVM2eoWsy/GoNu4XhksgrYA3QnLus98p5hBwB490+zAlqd3uPqmkYX
hw4F48uH7lxlqfI8X9oPL/d8V+PVkissbGm2X3cWZiCPjGKESamUgw77Ky77ckkv7jzgB0kyZrBn3pnW9
cv7+AEKKGde8Z01jCRsTRy29e/6ej3eBKM7Ry4aUbtVq06H0fG6oGMWlyyEku6Ricqp9apiPLFm94y820A
MybAwW+/5IoAywE8nFj/0nu/BzDyscZLM9y/15MQrR3sIiIg0C08h1tViuDUY78n1g9Mf/nFm03tPUCVWb
BQWtt2NHne3stCLA0RY6BSsLKZ48QWrEUeHI901SiDTKWJvnr9m1v/BXBGBdjztcvVTARAxCyKk78DoJul
nBi9tJiZunRZrWBlIwcAEdSE8XbKZDdGFaa/+Zu2Iq/KUVX0844jagyLcAx18GX30ax3Ga7lx7px3QWca
RVWUy0ig+v3HpDexdnkIjTYHkuA7XZGZtIUYY/AdB2DjqAJ20iDUFhVa0LE8EJrKh1WFbpMp1HTJdIRUtM
jRgFNRIInBrnNzuk9Yx+GMLx6YSJTtoITeyitFFeuv757A0BZEYAiIzZleTowptEzh84UWNpFI5ho2ePVzk
vH6maXNFLA8AKDaQArseAwNcwEs97q4BdW/dh0SqF0vbM7Jo9M33TJDtoAz6oA5AHy8omDx7yEooCyYEiV
KA+cSGI0tdQZqBQsr+dETgEGkFaGJ76yk777msddM9F06JYeB3tirzduumHm6/R4FgUocygC5AUAAjJtSK
fPlyVooNoNnD0Z0xpMaJEw1Y2pRIFGwQkLYiFgMC/AMIRO4U9V/PEoVRBhxHvb0Xp4+q09QctzIkBBkZUc
yeBkJ4JgAgg0ZIJ2PdaN0a5hmcJJcAlEy4xu3T9exaBKCAFDUAztrTzSqlG2AwYIdJ3jW7HYP298bPpzsg
Pl0aJ3F/jESjUQAAGTko0EoKASBataaTXUapZRd6o15mCedPKBTvuLxwYaJUIKFIDDYCgrlr1qz/e9sMs7
+Y/aU0Ubq7Yfm2YpcfCTzSs03N58zakY+cfm5TwNRj5Se9VD71v1o4feu/YbP73qN6782c3nfe7Bv1j9k3
0fHngHZzv330a852PNqal/iW3sjqrTu63xvY0316x1Z2T7PtYIj9zevIALiG0R0K+60gFqtSSQ+IAYQ0lk
nDp2Yi535dBZ/50Z/R9vvrCM/DlwuJHyxpmcDxn8auT4181vaV1/1jqgh62LvVads3063TAowro00sTMNg
GcpQL0JjxBkjetW1kwPGBV0viv6hVduW55hnj/4tFP1C86qWwWxuC5Lk0qvloIraZYXfmbNqpIrUlSr6LJ
Wd4EXTq4LKqkkNY4JQJRteKrdajUiapVrFYdYgkSsVjE1UHxCrFDECyOY5cCiSGJJ3RD86wWQOJomY8EYo
8XAR8lLnWQgEsFShs6q3sAEm2N0hiiCj5NMB8nPo0gSojT5NS66ex2gK+0pjqmU5k4ALzp6InYGQJmRFFo
seT4Jb/kfWHNd//LxCdb0AAAAABJRu5ErkJggg==" | base64 -d > image.png
```

```
[x]-[user@parrot]-[~]
$openssl enc -ARIA-128-CTR -K e7b72ae8cbdbff7b4a4ab3a6c3ab0b7d -iv 3e7275b2c6
d2c14fec87c685fb61814c -in image.png -out image.enc
[user@parrot]-[~]
$base64 image.enc > image.enc.b64
```

Po tym w pliku image.enc.b64 trzeba usunąć sobie enter-y bo inaczej nie ogarnia (można ręcznie)

```
[user@parrot]-[~]
$curl -X 'POST' 'http://127.0.0.1:2011/submit' -H 'accept: application/js
on' -H 'Content-Type: application/json' -d '{
  "session_id": "07544d15-674c-4ff9-bf70-4e260a468712",
  "encrypted_data": "'$(cat image.enc.b64)'"
}'
{
  "details": {
    "algorithm": "ARIA-128-CTR",
    "encrypted_size": 3586,
    "image_file": "postal_horn.png",
    "key_size": "128 bits"
  },
  "message": "\u2705 Gratulacje! Prawid\u0142owo zaszyfrowa\u0142e\u015b obrazek \
postal_horn.png\" u\u017cywaj\u0105c ARIA-128-CTR!",
  "success": true
}
```

2.12 Pod adresem <http://127.0.0.1:2012> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:2012/decrypt> oraz <http://127.0.0.1:2012/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 2012:2012 --name ex12 docker.io/mazurkatarzyna/symmetric-enc-ex12:latest
podman run -p 2012:2012 --name ex12 docker.io/mazurkatarzyna/symmetric-enc-ex12:latest
```

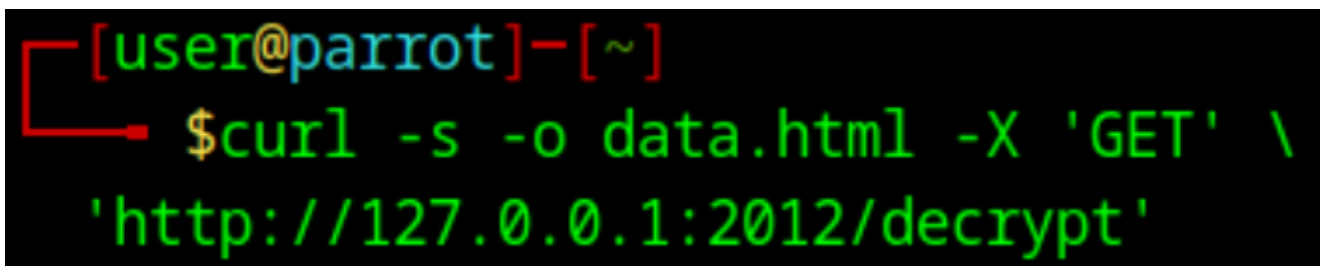
Jeśli Docker Registry nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 2012:2012 --name ex12 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex12:latest
podman run -p 2012:2012 --name ex12 ghcr.io/mazurkatarzynaumcs/symmetric-enc-ex12:latest
```

- (b) Wyślij żądanie HTTP GET do endpointa <http://127.0.0.1:2012/decrypt>. W odpowiedzi otrzymasz identyfikator sesji (`session_id`), obrazek do odszyfrowania (`image_data_base64`), klucz do odszyfrowania (`key_hex`) oraz IV (`iv`).
- (c) Do odszyfrowania obrazka użyj algorytmu ARIA-256-CFB (bez paddingu), wykorzystując odebrany od serwera klucz deszyfrujący oraz IV.
- (d) Wyślij do serwera odpowiedź poprzez endpoint <http://127.0.0.1:2012/submit>, używając metody HTTP POST, przekazując dane w formacie JSON: identyfikator sesji (`session_id`) i odszyfrowany obrazek zakodowany w `base64` (obrazek jest najpierw odszyfrowany, a potem zakodowany) (jako `decrypted_data`). Serwer zwróci w odpowiedzi informację o tym, czy obrazek jest prawidłowo odszyfrowany.
- (e) Zapisz odszyfrowany obrazek na dysku z rozszerzeniem `*.png`.

UWAGI:

- Aby odszyfrować wylosowany przez serwer obrazek, wykorzystaj narzędzie `OpenSSL`.
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.



```
[user@parrot]~$ curl -s -o data.html -X 'GET' \
'http://127.0.0.1:2012/decrypt'
```

ARIA-256-CFB

⚠️ Poniżej znajduje się zaszyfrowany obrazek zakodowany w Base64. Użyj podanych klucza i IV aby go odszyfrować.

ENCRYPTED IMAGE (Base64):

```
sRYEM1tVW1MbzJk8aTz3KacbvTG6H0i97bmOKo01P8u0MFd86cVL8M0kaB/g/Aak7/fBNAADILx07zRau2BGNJsqcgh1Gbe8g9CVYL/QpYm/K7x/qTza4Noqt0rDuaE+V1QrL/xDMJFd  
Lk10fZ1IhdGAPM+f/x090ID1JUT/X7JbCT1BLDqpb2c3zNLz7yA9eJys3+P7y7nPm0GwEac1H9KQKZEjQM0+aEuFip/1SLNAd8C6cF0WwJWb8/Cbn8J05+aqE5agudV17FH0WexymdDw  
+7pFCH8dKz1K1pVvTEJntzVv4V8Pp/JPM0YcQHfka5NMB+osotq2xNDUGCBuV1Axs9Mya1txd14Y5wo+Qvj4w9F4K7D6JZ34Y8M0WYA1F2E/pbxKw1IXPe100j5xykdF8Q8REnVYVYx  
arndQL5Yz78JkV8H11+ZakYI1W0y1v1k3L+A31P10zYs0+LCdnxjBw2HQV1qfW7yQ8L1dLMD000EucCAUNTYnE1wQWY+j*MB1Yy1CEaBxPp7Bwa1dAaYn519pJkA3X8u8K8FuAJp  
qkw1Qx1ex1VgQj/TNzypnT2zk173M0q27P+f+qtPyE1Kp2YX11W0Qp2B3tHPz5PH0vGekn10rRQRq1Js5Jw1GXdJZ188HIF3fWn1E1KYo1PouSPCn1V9R/1nMEfPKzusKtKXhYo8Qz75  
p1Zcy/53QxbsUfHfVn2F8/Ja9Mvpg2EQQU8z100Z/pyxxZakhWV+V8MEf5z8B9Vn18JvYjF518GQ00KtYn1Qtdh9BN3gBuY12XX9bUUCRw8hymd+y15NzoojT17jx51eQ8x7XnFz7oiz  
4G01tc3byq41Xo3b2D+7Q17FvVVMWu3Jh2IazbVFbuDpgF01s1+18h2N1/51dx55pryP8w80G104tF8KsD/g5TYWfawuZWLXQ4FF80zrX0hu/Y8u1pF9/v1f2FQVLP0gs8GZP+3LNaJ  
wMPHox2ZK1P6Y6uWu8RL1ZDxuXKQ2X7U146zpu8+1M0zGLJOG8pwGL4V75e+Myurn6sCQyc2KxhYf1DM25Favc5b2wFJQ28XZF8CjKCFJcQqxbFZ1K968zxd61TF1R0fXwZ85k58Dd7D  
j3k81HJ51XMM21FBN1D16KLVMJA14Vyy+58sKXKndTouSF2NWX1vdWVYQ61EHYqgFnfDEGcgJ1vwmJwqfyvFD15M9q8eepRQ2LPu088htzFyM01Lvj1IVFutks8fUp8Nnd6TeXc6G1fE  
8qWn17uutankZFaLAXcY11Q858uA1CvkaJ0g3qGChz03K78f5Gq88wG2vLYuY4zUhg/kxXAB359ok11LQhYMBPpf1A8NPVP7g0u88ZnPyX/Tgvgg2eN8bRP318N1eRJ8ztjVE7nhLm  
UNEX5y86U1F1K9u1PcUy02KCUhuc5Kb14e0EJ//XC8b0btgv2FXkmPTXLn11140M1FwKuVzA086tMTgzzXVTb8Rj6Bqkxwuln3YaCQz8WS5a8G8fEJf+hJ5zExx6P5BHL1LazQU  
z/8Q04E21A5Gruccosg44KQID10k+ziKco8n6KUbU9L75UP4qWvVz1b17Y8za5U83fhgA8McA4+CYuYw8+bxk8gwap1o1ctYn7+WA3j/Ky8w/z8c8Gj1x98Y8wC+ax7L4Kpqj18g28pk  
ULz9YyYMF178C3FF/wJLfbu4eY058g5cz8/X0D7J1N5b8zALzZFFCZfg02vLga08M39MFU/+LWch/n8wGExx18zJWYqD20dHjuQ0ysgFb9GLLw/zdJw8yFMZhdgFKeeRvYh9MA15TTpk  
Y4UACzt68K1K1+HcFm8w8k8cUmsdtABfz2uKJHh30fp+C8azY/wPit1wFH2V0Uj+9FEOE8Dg1YmVWCRjHMAQ0M1y5C0eFEEUMLZUzJ1PYMC10Gw/qF0pM8y11uPPQwu+YP91s8  
wdCxd7qKb1JF80qQ8CAFN2+VCP32FawMBE1CRQ/24951/wjQ+8cFz26o+uPn5wZ86Uo+mvWt2TC83hy8+ZF5eKcK21ndjN1Vcty6d7X0z1fvY3a+q8JFGQ8J1wVfaEK/dt6x3GEAe  
zyX7j/7A+8ZJG3qjxL81YaRVL1PousK654dGPMKGFq8W8PMKR91sgd1800unE25fz2xRhKp8v81msCKDFJq4aE82k18dQMw.9p4j9ZX4Qp71v4701/1E+aP1vqg3L5en1cpervwTzQLvzH  
txwM+8W8y5M2xwec9MP8Ld1Dz7/h3JtUdUf1ZQC+WzZuk31w5+Q28FdgM8tgsPE2PAJex040ccexh1E5VCEV8AyuFRhD2aDCdzIhm8jzZyZ5gpfz99CBQ8Q2hz84VwMa3NauV7LwUzthQ18w  
11w7Yfddvnx58h7bcYpQDL8e6KYR8Phz823p45vMxJzjzY08wNQ/P31dH5gGMucFg5Uhqg63ugn8YGC83hftGw8DTw1gr6Mkxy2eF31c5w8p10J1ds50pwb8BdZJFjHnEeFgo8Ko  
FNM8Kx5LQh8sp8jxG72D4H18DPu11s0pJ595tJQFV8KqP70w7qFwbpV/HV8WxziX8qFYzqDARh1a2YTLRu8FNy8zJfAvF615c9qK35UJZTUBAzv7GCy26/JvxFzq3K5jccokAoTR  
IMjxDrVNP8GzwVp1N1aCtY9cgn5qIqLy+Qu1k2jFzGw8LtbwCjyE5qt1jQh8RQh8ELM3bG8X80uALxUyJQ8wQ24AnqIJaRvdE101cEcPPEJPGATAsFpuy2KHIJyXz288v8mfw  
cG5R18xIUTjh58Lr8q/skrRaLr8uZjN2RCABQ==
```

SESSION ID:

ca88d993-81e3-4450-9bb5-984c4225f1c0

KEY (hex):

24eef8238d709c0c5858304803aab3f7717993e005b9a97921797120e4f00d0b

IV (hex):

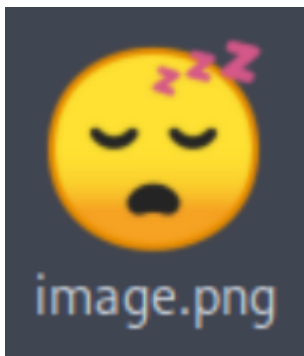
90508498085e7cd10f0785c4e3cdde9

- 🔑 Algorytm: ARIA-256-CFB
- 🔑 Rozmiar klucza: 256 bitów (32 bajty)
- 🔑 Rozmiar IV: 128 bitów (16 bajtów)
- 🔑 Rozmiar zaszyfrowanych danych: 2671 bajtów

```
[user@parrot]~  
$ echo "GZz7ST3Yw3K9JXJXvk6rhr80ijGDd4/XSaN0lqDN+1XxUyolG8nm82fPJ3hzVH0HjAfGX3  
fmuHcxWc2vmuva85FE5jYDJ3a9ePqK9GA0U4Rh3scXkeNlonwUNTVBe7xpkfy0lbg982s7DNAtDg6/jS1Y  
oGVgU8J2T0hotLYys7tNM9QisNnH7gN0j51N0npZuxmduOKv+16bs0QmFs9r5JI42Erp/gJ+1J/Oe0eljp  
GecCB32QLBKxaQqjGr2Y0VAN0zhCfbKY2pjbivQn0qYQqGiGctJQvSERqkpjq8BcAxxHZp2JngmhaM6mb  
V8ix/DHn2VmVwUjBT4FNsfa5JnzYwakxCWB8gK8/RZ2ELFYw7N7V0nUdfPrwlyzzBLJEKWLmr50cEKeE3y  
+eI5ct8Qxu12NWc0fVB8l4s1KrgGBiXhB3xZJYc8k+C2ZBiZANwhxEPGcYInqn4pXnh1VLTdGq/fUoI+0/  
1xn8DcfvqrEAA8vfRdyMWzchDxkbytrI06oJCnR7d4mbcAMHTwm6aKU30LQ01M06xMzh8L3cEydsWg3kc7  
83q/aTtFJBazZqo7t4crUu9qgvKyJt6iynGhhAanVAP2Rati/bF3rA+5dym7gWEgHdDq50X18Z14IZNYA  
lAGLnt/U2eIVlsFRs+Lq+b59XmOPhOzfgHKak/nBg7RKkN8jOHvwEHAtJd1V0sRYEM1tvWiM8zjk8aTz3K  
acbVTG6H0iq7bmOKo01P0UoMFd86cVL8MGWawB/g/Aak7/f8NAADILx07zRau2BGNJsqcqHlGbe8g9CVYL  
/QpYm/K7s/qTza4NogtDrDuaE+VIQrL/xDHJfDLk10fZ1IHdOAPM+f/k090IDljUT/X7JbCTi8LDApb2c3  
rNLr7yA9eJyi3+P7y7nPsm6GeEsciHKkQN2EjQMd+aEufip/lSNiAdMC6cfUHWJMm0/CHn8JD5+xQE5aqu  
dV17fHGwWxynEDW+7pFCHRDkzbIk1pvYEjntzvVdXV0PrJPW0YCWQHZFkmShNb+osotq2xNDUGCRuV1Axx  
MMyaItxdI4Y5wo+Qvj4w9F4K706JZ34Y8NKWYAIF2E/pbxKeiIXPmI00jSsykdf8QHREWFVYXxann0L5Yr  
78JkvHUt1+EzKY8IWV0yIvIk3L+A3iP10zYs0+LCdnhxjNe9HQYiqfmW7vGNLldLHMDD09EucCAUNTYnEi  
wOwy+j+bEb1YytCEmBxPp7BWaidAzYn5t9pjKaXkHeu0K0fuAJPqokwIQxlexlvGq/TNrypnT2zki73MUq  
27Pf+qtpyEIKp2YXLlWEQp2B3tHPr5PM5vGeXmIOrrQRqiJsSjw1GXdJ2l88HIF3feNiEIKYo1PsuSPCnl  
Y9R/inWfEfPkruSwtMXhYo8hQr7SpIZcy/53QxbsUfHVM2EB/Ja9MPvg2EQUQj0rI0BZ/pysxZWkhMV+V0  
MEf5z0BMVW18jVYjrF510GQDDKtYn1QTdh9BN3gBuY12XX9bUUCRWWhyn6+y1UNzoojTI7jsSieQBx7XnF  
z7oiz4G61tc3byq4iXo3b2D+7Qi7FvkVhM/w3jh2ImrbVFbuEpgF0isi+lRh2NI/5ldsSSpryP0eb9D104  
ifRmKsD/gSTYfwewZWlXQ4FF00rrHXhu/Y6uIpF9/v1fIFQYLP6gs8GEP+3LNajwMPbxb2ZKIP6Y6uNw8  
RLiZDxuXKQ2X7Ui46rpu8+lM2rGLj0GRpw9L4V75m++Wyuurn6sCQyc2WrhYyIEM25FavcSb2wFJQ2BXZF8  
CjkCFJcQqxbFZIk960rzd6lTfIRofXeZR5k5RDd7Dj3k0lHJ5IkWw2LF8WldiWLYMJat4Vyv+58sWXXkHd  
TOuSFZNVXlvdNVfQGIEhYqgFnfDEGCgJlvaWJeqHyvFD1SM9q0eepKQ2LPu080htrEFyMpiLVji1VFutks  
BfUp6Nm86TeXc6GtfE8qWmi7uutanWZFaLAXcYIIQ85BuAICvkaJOq3qGcbz03M70f5Gq0B8wGZvLYu8Y4  
zUhG/kxXA03S9okliLOHpM0PdfLA0NPVP7g6uBBZhPyX/Tgsqg2eENRbRP5i8N1rMJ6ztjVE7nhLmUN0XS  
y06UlfIkNw1PcUyBD2KCUNuscGkbI4mBEJ/XC8b0btgv2FXkmPTXlnii4DMLfwsKwVzAOR6tMTgzzXVT  
b0Rj6BqmXmWuin3YaCQR8WSSa0G0fIEjf+hJSzEXxs6P5BHLILmzQUz/0Q9o4E1A5Grucooig44KQIDiBk  
+ziKco6n6kUbuH9L7SUP4qNvVr1b17Y8za5U03fhgAaMcA4+CYuYe8+bsk0geaploIctYn7+WA3j/KyBw/  
r8c0Gjls90YBwC+ax7L4XpqjsNg28gkULz9YyYNft78C3ff/eJLfbu4eY05BgScz0/XOD7jN1Sb0zALrZf  
fCZfg92vLga08MN3HWFU/+LWch/n8wgExXr8rjWYq02UdWjuQYsgfb9GLLw/zdJwByFHZhdgEKmbRxYhS  
MA15TTpkY4UAGztd6BK1KL+HcfNwmvhwk0cUmxdtARfrZsuKGJHh30fp+C8arY/mPiTlmFH2V0uj+9FE0E  
kDg1YMwVWCRjhMAM2GM1y5C0efEEUMLZUrJIPYWCi0Ge/oF0pW8yi1uPPQnu+YP9Is8adCx7qKbIJF00q
```

```
QHGAfN2+VCp52FamxMBELGRQ/249S1/wjQ+8cFc2Go+uPn5wZh06Uo+nvvMt2TC03hy0+ZFSmXmSM2indj  
H1Vrty6d7XDzlfvY3a+qRjFGQRjIeVfaEK/dt6xXGEAezyX7j/7A+BZJG3qjxL01Ya8VL1Peusk654d9PM  
KGFqBWBHkr9isgdiB00unEZSfz2xRhKpBv0IlmscKDFJq4aE02kl0dQMwL9p4j9ZX4Qp7lv470i/iE+aPI  
vqg3LSmicpevwVtrQLvrHtsww+4W6ySWZxeec9MPBLdIOz7/h3JtUdUftZQC+WzZuk3lwS+Q2BFdqMnEgs  
PE2PAJWx040cexhiESVCEV8AyuFRhD2aDCdzIhm8jrZyZ5gpEz9SCBQN2hzW4VeMa3NaaV7LeUrhQ18Wl1  
re7yftdnvxSBs7hCYpQOL9m6cKYRePhzb23pw5vMxJzjrYONwsNQ/P3XdM5gGMwCfGsuHqq63ugnbYgCM3  
hftGw80Twlgz6MksyZeE31c3WmVp1DJIdS0pwobBbEZJfjHnEefgo8KoFNM8Xk5LQkHsp0jxG72D4HiBD  
Pu1IsDpj59StJQFVBDKqP79w7qfWbypV/hV8MxrIXbgfYrqDARhAlm2YTLRuufNyjezJfAvfGiSc9qK3SU  
UZTUBAzv7GCy26/JsvFrq3K6jco+okAoTRIMjxDzVNPGPGzwwVp1N1aCtY9cgrn5qZqLy+Ou1k2jfrGsw4  
LtbwoCjyESqtljQh0RQwH9ELM3bGbX00uALxXuYJQNuWQ24AnqIJmRvdEL01cEcPPEJPG8TAsFpuy2KHIIJ  
yXz20BvRmfwcGSRiBsIUTjh50LrBq/skzRaLr8uZjNZRCABQ==" | base64 -d > image.enc
```

```
[user@parrot]~  
$ openssl enc -d -aria-256-cfb -K 24eef8238d7696cc5858304803aab3f7717993e005b9  
a97921797120e4f00d0b -iv 905684986085e7cd10f0785c4e3cdde9 -in image.enc -out image  
.png
```



Gdy wykonamy to polecenie, plik image.b64 będzie miał nowe linie

```
[user@parrot]~  
$base64 image.png > image.b64
```

Gdy wykonamy to polecenie, plik nie będzie miał nowych linii i przejdzie bez edytowania w serwerze

```
[user@parrot]~  
$base64 -w 0 image.png > image.b64
```

```
[user@parrot]~  
$curl -X 'POST' 'http://127.0.0.1:2012/submit' -H 'Content-Type: application/  
json' -d '{"session_id": "ca88d993-81e5-4450-9bb5-984c4225f1c0","decrypted_data":  
"$$(cat image.b64)"' }'  
{  
  "details": {  
    "algorithm": "ARIA-256-CFB",  
    "decrypted_size": 2671,  
    "image_file": "sleeping.png",  
    "key_size": "256 bits",  
    "original_size": 2671  
  },  
  "message": "\u2705 Gratulacje! Prawid\u0142owo odszyfrowa\u0142e\u015b obrazek \\  
\"sleeping.png\" u\u017cywaj\u0105c ARIA-256-CFB!",  
  "success": true  
}
```